End-Sem: CS-2203 Artificial Intelligence Lab

Assignment No: End-semester Exam

Date: April 16, 2026

Submission Deadline: [April 16, 2026 - 17:45 Hrs. IST]

Instructions for Submission

General Instructions

Some assignments may need to be submitted via Google Form. Please carefully follow the instructions below to ensure proper submission:

File Preparation

Submission Instructions:

1. Store all assignment-related files in a single folder and compress it into a `.rar` or `.zip` file. The compressed file must be named in the following format:

   `roll-number_ese.rar` or `roll-number_ese.zip`

2. For example, if your roll number is `2201AI01`, the compressed file should be named as:

   `2201AI01_ese.rar` or `2201AI01_ese.zip`

3. Save each program using the filename format specified alongside each question.

4. If all assignments are completed within a single Jupyter Notebook, name the notebook file using the following format:

   `CS2203_ese.ipynb`

5. When using a single notebook for multiple problems, clearly **segregate the code for each problem**. Each problem must be introduced using a **text cell** containing the problem statement before the corresponding code segment.

6. After completing the notebook, place it inside the assignment folder, compress the folder, and upload it by following the above naming instructions.

Note on Academic Integrity and Plagiarism

Note*: All submitted source codes will be evaluated using advanced plagiarism detection software and AI-generated content analysis tools. This includes checking for copied code, unauthorized collaboration, and excessive reliance on AI tools without proper understanding.

Students are expected to submit original work. Any submission found to be copied, partially copied, or automatically generated without meaningful modification may face mark deductions as outlined below:

- **Similarity below 10%:** Acceptable. No marks will be deducted.
- **Similarity between 10% – 25%:** Minor overlap detected. Up to **10% of the assignment marks** may be deducted.
- **Similarity between 25% – 40%:** Significant overlap detected. Up to **30% of the assignment marks** will be deducted.
- **Similarity between 40% – 60%:** High level of plagiarism. The assignment may receive **zero marks**, and the case may be reported for academic review.
- **Similarity above 60%:** Considered a severe academic integrity violation. The submission will receive **zero marks** and may be subject to disciplinary action as per institute guidelines.

Students may be asked to explain their code during a viva or demonstration. Failure to adequately explain the logic and implementation will be treated as evidence that the work is not original.

Submission Link

Upload your compressed file using the following Google Form link:

`https://forms.gle/iBWKnzjosHvMXWRF6`

Problem statement: Operation Rescue — The Maze of Miragpur [50 Marks]

During a severe flood in the city of **Miragpur**, a rescue drone must navigate from the **Command Center (node S)** to the **Flood Shelter (node G)** through a network of intermediate checkpoints. Each directed path between two checkpoints has an associated **traversal cost** measured in fuel units. The drone's onboard navigation system provides a **heuristic estimate** $h(n)$, representing the estimated fuel required to reach the goal G from any given node n.

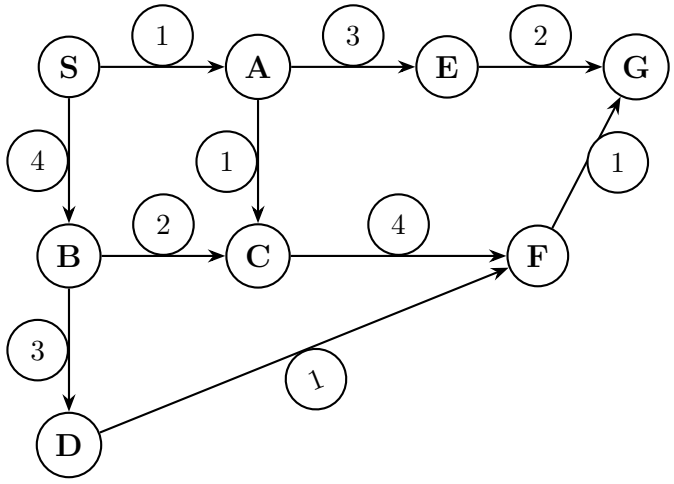
Objective: Implement the **A* Search Algorithm** to determine the optimal (minimum-cost) path from S to G.

At each iteration of the algorithm, clearly document the following:

- The node selected for expansion and the justification for its selection.
- The values of $g(n)$, $h(n)$, and $f(n) = g(n) + h(n)$ for each node in the OPEN list.
- The updated contents of the **OPEN** and **CLOSED** lists.

Graph Representation

The city's checkpoint network is modelled as a **weighted directed graph** with 8 nodes and 10 edges, as illustrated below:



Adjacency List (Directed Edges)

```
S -> A (cost 1),  B (cost 4)
A -> E (cost 3),  C (cost 1)
B -> C (cost 2),  D (cost 3)
C -> F (cost 4)
D -> F (cost 1)
E -> G (cost 2)
F -> G (cost 1)
```

Heuristic Table $h(n)$:

Node	S	A	B	C	D	E	F	G
$h(n)$	7	5	4	4	3	2	1	0

Tie-Breaking Rule

When two or more nodes have the **same** $f(n)$ value, break ties by preferring the node with the **higher** $g(n)$ value (i.e., the node that is deeper/closer to the goal).

Tasks

(a) **[10 Marks]** Apply the A* algorithm to solve this problem in an efficient way.

(b) **[5 Marks]** Report the optimal path discovered by the algorithm and its total cost.

(c) **[10 Marks]** List **all** possible paths from S to G with their respective costs. Verify that the path found by A* is indeed optimal.

Bonus Task:

4. **[Bonus: 10 Marks]** Present a complete step-by-step trace of the algorithm showing, at each iteration:
   - The node selected for expansion and the reason for its selection.
   - The computed values of $g(n)$, $h(n)$, and $f(n)$ for all nodes currently in the OPEN list.
   - The updated contents of the OPEN and CLOSED lists after expansion.

Input Format

The input to the program shall conform to the following structure:

Line(s)	Description
Line 1	Two space-separated integers N and E, where N is the number of **nodes** and E is the number of **directed edges** in the graph.
Lines 2 to E+1	Each line contains three space-separated values: `source destination cost`, representing a directed edge from `source` to `destination` with the specified integer traversal cost.
Line E+2	Two space-separated tokens representing the **start node** and the **goal node**.
Line E+3	A single integer H, denoting the number of heuristic entries (equal to N).
Lines E+4 to E+3+H	Each line contains two space-separated values: a `node` label and its corresponding heuristic value $h(n)$ (integer).

Output Format

The program shall produce the following output:

Line(s)	Description
Line 1	The optimal path, displayed as a sequence of node labels separated by "`->`".
Line 2	The total cost of the optimal path (integer).
Line 3	The ordered list of nodes expanded during the search execution.
Line 4 onwards	All distinct paths from S to G with their respective costs, clearly marking the optimal path.

Expected Output

```
======================================================
  A* SEARCH -- OPERATION RESCUE: MIRAGPUR
======================================================
  Optimal Path  : S -> A -> E -> G
  Total Cost    : 6
  Nodes Expanded: S, A, E, G
  Expansions    : 4
======================================================
  PATH  PASS  (S -> A -> E -> G)
  COST  PASS  (6)

--- Alternate Path Costs (brute force verification) ---
  S -> A -> E -> G               cost = 6 <-- OPTIMAL
  S -> A -> C -> F -> G          cost = 7
  S -> B -> D -> F -> G          cost = 9
  S -> B -> C -> F -> G          cost = 11
```

Evaluation Criteria 1 : Total 35

(a) **Correctness of Implementation (40%):**

- Accurate Functional Python implementation of the A* search algorithm.

(b) **Code Quality & Q&A (30%):**

- The code should be well-structured, properly commented, and follow standard Python programming practices.
- Students may be asked questions related to their implementation to assess their understanding.

(c) **Error Handling (10%):**

- Code correctly handles tie-breaking, path reconstruction, and edge cases.

(d) **Documentation (10%):**

- The code is accompanied by appropriate documentation and comments that explain the steps taken in each part of the program.

(e) **Timeliness of Submission (10%):**

- The assignment is submitted on time, adhering to the submission deadline.

Evaluation Criteria 2: Total 15

The evaluation of this lab assignment will be based on the following criterion:

(a) **Viva (15 Marks):**

- Questions will be asked related to the assignment to assess the student's understanding of the implemented concepts.